Object-Oriented Programming (2)

COMP1039 Programming Paradigms

Yuan Cheng

Spring 2026

School of Computer Science
The University of Nottingham Ningbo China

Object-Oriented Programming (cont.)

The `static` Trap (predict)

ThisTest.java

```
1  class ThisTest {
2      int a, b;
3
4      public static void main(String[] args) {
5          a = 10; // what will happen?
6      }
7  }
```

The `static` Trap

Why does this fail?

ThisTest.java

```
1  class ThisTest {
2      int a, b;
3
4      public static void main(String[] args) {
5          a = 10; // compile error
6      }
7  }
```

- `main` is **static**: it has no `this`.
- Instance variables must be accessed through an **object**.

Correct

```
1  ThisTest tt = new ThisTest();
2  tt.a = 10;
```

`static` Keyword

- Normally, variables/methods are accessed through an **object** of its class.
- `static` defines a class member used independently of objects.
- Static variables are shared by *all* instances of the class.
- General form: `ClassName.variableName`

Example: Static Variable

StaticDemo.java

```
1  class StaticDemo {
2      int x;
3      static int y;
4      int sum() {
5          return x + y;
6      }
7  }
8
9  public class SDemo {
10     public static void main(String[] args) {
11         StaticDemo obj1 = new StaticDemo();
12         StaticDemo obj2 = new StaticDemo();
13         obj1.x = 10;
14         obj2.x = 20;
15         StaticDemo.y = 19;
16         System.out.println("sum obj1: " + obj1.sum());
17         System.out.println("sum obj2: " + obj2.sum());
18     }
```

Example: Counter

CounterDemo.java

```
1  class MyClass {
2      static int count;
3
4      MyClass() {
5          count++;
6      }
7  }
8
9  public class CounterDemo {
10     public static void main(String[] args) {
11         for (int i = 0; i < 3; i++) {
12             MyClass obj = new MyClass();
13             System.out.println("Class " + MyClass.count);
14         }
15     }
16 }
```

Static Methods (and Restrictions)

- Call form: `ClassName.methodName();`
- Useful for utility functions (e.g., `Math.sqrt()`, `Math.cos()`).
- Restrictions in a static method:
  - can directly access only static data
  - cannot use `this`
  - cannot directly access instance fields without an object

Static Error (Fix by Creating an Object)

StaticError.java (wrong)

```
1  public class StaticError {
2      int x = 3;
3      static int y = 1024;
4      public static void main(String[] args) {
5          int z = y / x; // error: x is not static
6      }
7  }
```

StaticError.java (fixed)

```
1  public class StaticError {
2      int x = 3;
3      static int y = 1024;
4      public static void main(String[] args) {
5          StaticError se = new StaticError();
6          int z = y / se.x;
7          System.out.println(z);
8      }
9  }
```

Controlling Access to Class Members

- Encapsulation links data with the code that manipulates it.
- It also enables control of access to class members.
  - e.g., prevent misuse of the data
- Example misuse: external code setting `mpg` to a negative number.

Access Modifiers

Access Modifiers controls the member access:

- `public`: accessible from any other code
- `private`: accessible only inside the same class
- `protected`: useful when inheritance is involved

Rule of thumb: Make fields `private`. Expose behavior via methods.

Private Fields Prevent Misuse

Vehicle.java

```
1  public class Vehicle {
2      private int passengers, fuelCap, mpg;
3
4      public Vehicle(int p, int f, int m) {
5          this.passengers = p;
6          this.fuelCap = f;
7          this.mpg = m;
8      }
9  }
```

VehicleDemo.java

```
1  public class VehicleDemo {
2      public static void main(String[] args) {
3          Vehicle car = new Vehicle(4, 14, 12);
4          car.mpg = 20;
5      }
6  }
```

Encapsulation via Public Methods

Vehicle.java (public getters; private helper)

```
1  public class Vehicle {
2      private int passengers, fuelCap, mpg;
3
4      public Vehicle(int p, int f, int m) {
5          this.passengers = p;
6          this.fuelCap = f;
7          this.mpg = m;
8      }
9
10     public int getMpg() {
11         return mpg;
12     }
13
14     private int range() {
15         return mpg * fuelCap;
16     }
17 }
```

Encapsulation Pitfall: Leaking Internal References

A and B

```
1  class A {
2      public String name;
3      public A(String n) { name = n; }
4      public void printA() { System.out.println(name); }
5  }
6
7  class B {
8      private A a;
9      public B(A a) { this.a = a; }
10     public A getA() { return this.a; }
11 }
```

Question: Is `private A a;` sufficient if `getA()` returns the reference?

Encapsulation Pitfall: Demo

Test.java

```java
public class Test {
    public static void main(String[] args) {
        A a1 = new A("A1");
        B b1 = new B(a1);

        A a2 = b1.getA();
        a2.printA();
        a2.name = "A2";
        b1.getA().printA();
    }
}
```

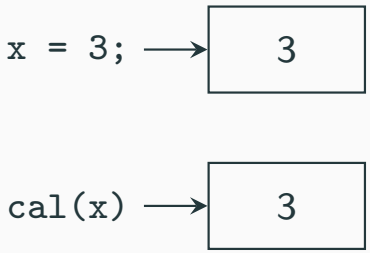
Takeaway: Encapsulation can fail if you expose mutable internal objects.

Why: Passing Arguments in Java

- **Two classic parameter-passing models:**
  - **Call-by-value:** the *value* of an argument is copied into the subroutine
  - **Call-by-reference:** a *reference* to an argument is passed to the subroutine

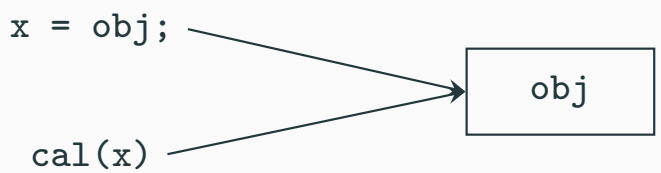
- Java is **pass-by-value**:
  - for primitives: value is copied
  - for objects: the *reference value* is copied
- Result: methods can mutate the object **through the copied reference**, but rebinding the parameter does not affect the caller.

Primitive argument
(value copied)



call-by-value

Object argument
(reference value copied)



"call-by-reference" effect

Example: Primitive Argument

TestPass.java

```
1  class TestPass {
2      static void make3(int x) {
3          System.out.println("(2) " + x);
4          x = 3;
5          System.out.println("(3) " + x);
6      }
7
8      public static void main(String[] args) {
9          int x = 4;
10         System.out.println("(1) " + x);
11         make3(x);
12         System.out.println("(4) " + x);
13     }
14 }
```

Predict: What prints at (1)(2)(3)(4)?

Example: Object Argument (Mutation)

TestRef.java

```
1  class Data {
2      int x;
3      void addTo(Data d) {
4          d.x = d.x + this.x;
5          System.out.println("(1) " + d.x);
6      }
7  }
8
9  public class TestRef {
10     public static void main(String[] args) {
11         Data d1 = new Data();
12         Data d2 = new Data();
13         d1.x = 3; d2.x = 4;
14         d1.addTo(d2);
15         System.out.println("(2) " + d2.x);
16     }
17 }
```

Trick Question: Rebinding the Parameter

What prints now?

```
1  class Data {
2      int x;
3      void addTo(Data d) {
4          d = new Data(); // rebind local parameter
5          d.x = 10;
6          System.out.println("(1) " + d.x);
7      }
8  }
9  public class TestRef {
10     public static void main(String[] args) {
11         Data d1 = new Data();
12         Data d2 = new Data();
13         d1.x = 3; d2.x = 4;
14         d1.addTo(d2);
15         System.out.println("(2) " + d2.x);
16     }
17 }
```

Inner Classes

- Nested classes:
  - non-static nested classes: **inner classes**
  - static nested classes
- Why: grouping classes for readability and maintenance
- Scope: bounded by outer class; inner class can access outer members

General Form:

```
1 class Outer{
2    class Inner{
3    }
4 }
```

Inner Class Example

Outer.java

```java
class Outer {
    int x = 10;

    class Inner {
        int y = 5;
        void printX() {
            System.out.println(x); // can access outer field
        }
    }

    int addition() {
        Inner inner = new Inner();
        return x + inner.y;
    }
}
```

Creating an Inner Object from Outside

Test.java

```
1  public class Test {
2      public static void main(String[] args) {
3          Outer outer = new Outer();
4          Outer.Inner inner = outer.new Inner();
5          System.out.println("(" + outer.x + "," + inner.y + ")");
6      }
7  }
```

Question: Why do we need `outer.new Inner()`?

Method-Local Inner Class

- In Java, an inner class can be defined **inside a method**.
- Its scope is limited to that method (like a local variable).
- It cannot be accessed outside the method.

```
1  class LocalInner {
2
3      void aMethod() {
4          // Method-local inner class
5          class Inner {
6              void display() {
7                  System.out.println("Inside method-local inner
                       class");
8              }
9          }
10         Inner inner = new Inner();
11         inner.display();
12     }
13 }
```

Method Overloading

- Polymorphism: Redefine the way a class works by changing how it works or by changing the data.
  - Overload
  - Override
- Overload: Two or more methods in the same class share the same name, but have different parameter lists.
- Restrictions:
  - number and/or types of parameters must differ
  - return type alone is not enough to overload

Overloading Example

Overload.java

```
1  class Overload {
2      void ovlDemo() {
3          System.out.println("No parameters");
4      }
5      void ovlDemo(int a) {
6          System.out.println("One parameter: " + a);
7      }
8      int ovlDemo(int a, int b) {
9          System.out.println("Two parameters: " + a + " " + b);
10         return a + b;
11     }
12 }
```

Overloading Constructors

MyClass.java

```
1  class MyClass {
2      int x;
3
4      MyClass() {
5          System.out.println(x);
6      }
7      MyClass(int i) {
8          x = i;
9          System.out.println(x);
10     }
11     MyClass(int i, int j) {
12         x = i + j;
13         System.out.println(x);
14     }
15 }
```

Overloaded Constructors (Copy Constructor Example)

- Constructors can be **overloaded** (same name, different parameters). One object can be used to initialize another object.
  - This pattern is called a **copy constructor**.

Example

```
class Summation {
    int sum;
    // Constructor 1: compute sum from 1 to num
    Summation(int num) {
        sum = 0;
        for (int i = 1; i <= num; i++) {
            sum += i;
        }
    }
    // Constructor 2: copy constructor
    Summation(Summation ob) {
        sum = ob.sum;
    }
}
```

Overloaded Constructors (Copy Constructor Example) (cont.)

```
1  class SumDemo {
2      public static void main(String[] args) {
3          Summation s1 = new Summation(5);
4          Summation s2 = new Summation(s1);
5
6          System.out.println("s1.sum: " + s1.sum);
7          System.out.println("s2.sum: " + s2.sum);
8          System.out.println(s1 == s2);
9      }
10 }
```

Key idea: Same constructor name, different parameter list $\rightarrow$ **constructor overloading**.

Recursion

- A method can call itself.
- Needs:
  - **base case** (termination)
  - **recursive step** (reduces towards base case)

Recursion Example

```
1  void drawStars(int n) {
2      if (n == 1) {              // base case
3          System.out.println("*");
4      } else {
5          System.out.print("*");
6          drawStars(n - 1);      // recursive step
7      }
8  }
```

Recursion vs. Iteration

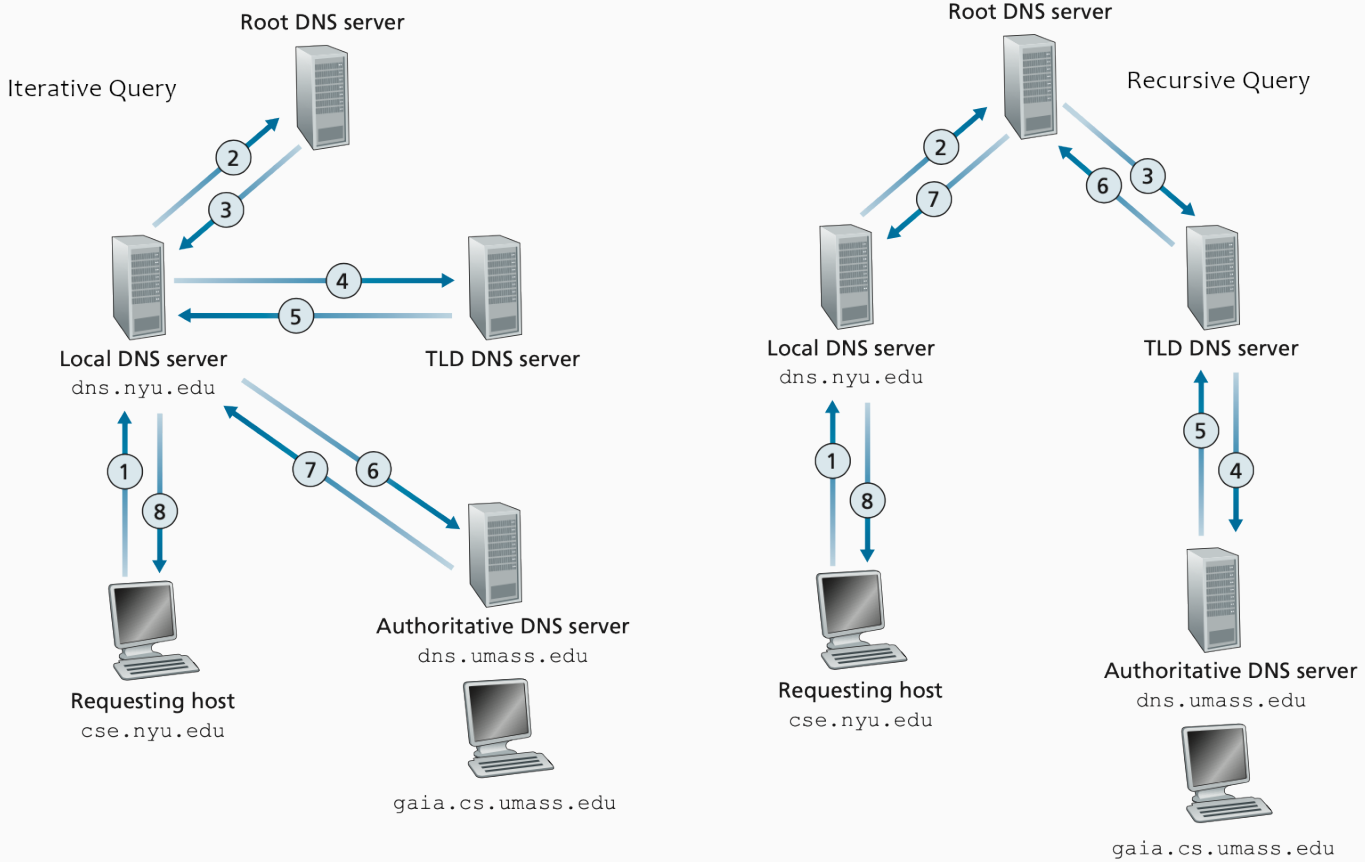
Iteration version

```
1  void drawStars(int n) {
2      for (int i = 0; i < n; i++) {
3          System.out.print("*");
4      }
5      System.out.println();
6  }
```

- Recursion often reads more clearly.
- But each call adds overhead (stack frames).

Iterative vs. Recursive DNS Queries

Assume that a user is trying to visit gaia.cs.umass.edu, but his browser doesn't know the IP address of the website.



Arrays

Arrays

- An **array** is a collection of variables of the **same type**
  - Arrays have a **fixed size** (cannot change after creation)
  - Elements are accessed using an **index** (starting at 0)
  - Arrays can be **one-dimensional** or multi-dimensional
  - In Java, arrays are **objects** (stored on the heap)

General Form (1D Array)

```
type[] arrayName = new type[size];
```

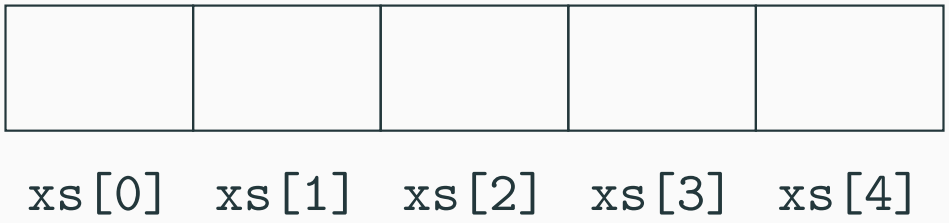
Examples

```
int[] numbers = new int[3];
String[] names = new String[5];
```

Arrays

- Each element in an array is accessed by its **index**
- Index starts from **0**

```
1 int[] xs = new int[5];
```



- Value assignment:

```
1 xs[3] = 5;
```

- Assign values using a loop:

```
1 for (int i = 0; i < 5; i++)
2     xs[i] = i;
```

Arrays

- Difficult to create and assign values one by one:

```
1 int[] xs = new int[4];
2 xs[0] = 1;
3 xs[1] = 3;
4 xs[2] = -13;
5 xs[3] = 20;
```

1	3	-13	20

- Alternatively, we can use initialization syntax:

```
1 int[] xs = new int[]{1, 3, -13, 20};
2 int[] xs = {1, 3, -13, 20};
```

- Length of an array:

```
1 xs.length;
```

Example: MinMax Problem

Given an array of integers:

```
1 int[] nums;
```

How can we find:

- The **maximum** value in the array?
- The **minimum** value in the array?

Example:

```
1 int[] nums = {4, -2, 15, 7, 0};
```

Expected result: Max = 15, Min = -2

Key Question:

- How many times do we need to examine the elements?
- Can we solve this using one loop?

MinMax — First Version

```
1  class MinMax {
2      public static void main(String[] args) {
3          int[] nums = new int[10];
4          int min, max;
5
6          nums[0] = 99;
7          ... // Array initialization omitted.
8          nums[9] = 49;
9
10         min = max = nums[0];
11
12         for(int i = 1; i < 10; i++) {
13             if(nums[i] < min) min = nums[i];
14             if(nums[i] > max) max = nums[i];
15         }
16         System.out.println("min and max: " + min + " " + max);
17     }
18 }
```

MinMax — Using Array Initializer

```java
class MinMax2 {
    public static void main(String[] args) {
        // Array initializer
        int[] nums = {
            99, -10, 100123, 18, -978,
            5623, 463, -9, 287, 49
        };

        int min, max;
        min = max = nums[0];

        for(int i = 1; i < nums.length; i++) {
            if(nums[i] < min) min = nums[i];
            if(nums[i] > max) max = nums[i];
        }
        System.out.println("Min and max: " + min + " " + max);
    }
}
```

2D Arrays: Array of Arrays

Example

```
1  int[][] table = new int[3][4];
2
3  for (int i = 0; i < 3; i++) {
4      for (int j = 0; j < 4; j++) {
5          table[i][j] = (j * 4) + i + 1;
6      }
7  }
```

In Java, a 2D array is an array of references to 1D arrays.

Irregular Arrays (Jagged Arrays)

Irregular initialization

```
1 int[][] table = { {1,2,3}, {5,6,7,8}, {9} };
```

- Java allows different row lengths.
- You can allocate only the first dimension:

Allocate first dimension only

```
1 int[][] table2 = new int[3][];
```

Length of a 2D Array

Example

```
1  int[][] table = { {1,2,3,4}, {5,6,7,8}, {9,10,11,12} };
2  int l = table.length;       // number of rows
3  int m = table[0].length;    // number of columns in row 0
```

Quick check: What are `l` and `m` here?

Why Not Just Arrays?

- Arrays have fixed size once created.
- Sometimes we do not know how many items we will store.
- Use `java.util.ArrayList`: a dynamic array abstraction.

ArrayList: Basic Syntax

Declare

```
1 import java.util.ArrayList;
2
3 ArrayList<Type> name = new ArrayList<>();
```

- `Type` must be a reference type (e.g., `Integer` not `int`)
- Provides methods to add/remove/query elements

Useful ArrayList Methods (Mini Table)

Method	Meaning
`add(e)`	append element
`add(i,e)`	insert at index
`get(i)`	get element
`remove(i)`	remove at index
`contains(o)`	membership test
`size()`	number of elements
`clear()`	remove all elements

Question: Which operations are easy/hard for arrays vs ArrayList?

ArrayList Example

ArrayListDemo.java

```
1  import java.util.ArrayList;
2
3  public class ArrayListDemo {
4      public static void main(String[] args) {
5          ArrayList<Character> a1 = new ArrayList<>();
6
7          a1.add('A');
8          a1.add('B');
9          a1.add(1, 'C'); // A C B
10         a1.remove(2);   // remove B
11
12         System.out.println(a1);
13         System.out.println("size = " + a1.size());
14     }
15 }
```

Exit Slips

- Take **1–2 minutes** to reflect on this lecture.
- On a sheet of paper, write:
  - **One thing you learned** in this lecture
  - **One thing you didn't understand**